COP290 Assignment 2

Task Board

A Jira-Inspired Project Management Tool

Design Decisions & Technical Report

Rohit Meena (2024CS10030) Subarno Saha (2024CS50431)

GitHub: https://github.com/88subarno88/Task_Board

March 2026

Contents

1 Project Overview **2**
1.1 Tech Stack . . . 2

2 UI Design Decisions **2**
2.1 Authentication Pages . . . 2
2.1.1 Login Page . . . 2
2.1.2 Registration Page . . . 3
2.2 Dashboard / Home Page . . . 4
2.3 Projects List Page . . . 5
2.4 Project Detail Page — Boards Tab . . . 5
2.5 Kanban Board Page . . . 6
2.6 Issue Detail Modal . . . 7
2.7 Project Detail — Stories Tab . . . 8
2.8 Story Issue Detail Modal . . . 9
2.9 User Profile Page . . . 9
2.10 Dark Mode . . . 10

3 Backend Design Decisions **10**
3.1 Authentication Architecture . . . 10
3.2 Role-Based Access Control (RBAC) . . . 11
3.3 Issue Workflow & Status Transitions . . . 11
3.4 Story Auto-Status Derivation . . . 11
3.5 WIP Limits . . . 11
3.6 Notification System . . . 11
3.7 @Mention System . . . 12

4 Database Schema Summary **12**

5 Testing Summary **12**

6 Setup Instructions **13**
6.1 Backend 13
6.2 Frontend 13
6.3 Running Tests 14

1 Project Overview

Task Board is a full-stack Jira-inspired project management application that enables teams to plan, track, and collaborate on work. The system supports Kanban-style boards with drag-and-drop functionality, issue tracking, real-time notifications, and role-based access control.

1.1 Technology Stack

Layer	Technology
Frontend	React 18 + TypeScript + Vite
Backend	Node.js + Express + TypeScript
Database	PostgreSQL via Prisma ORM
Authentication	JWT (Access + Refresh Tokens)
Styling	CSS Modules + Custom Themes
Testing	Jest + Supertest (100+ tests)

Table 1: Technology Stack

2 UI Design Decisions

2.1 Authentication Pages

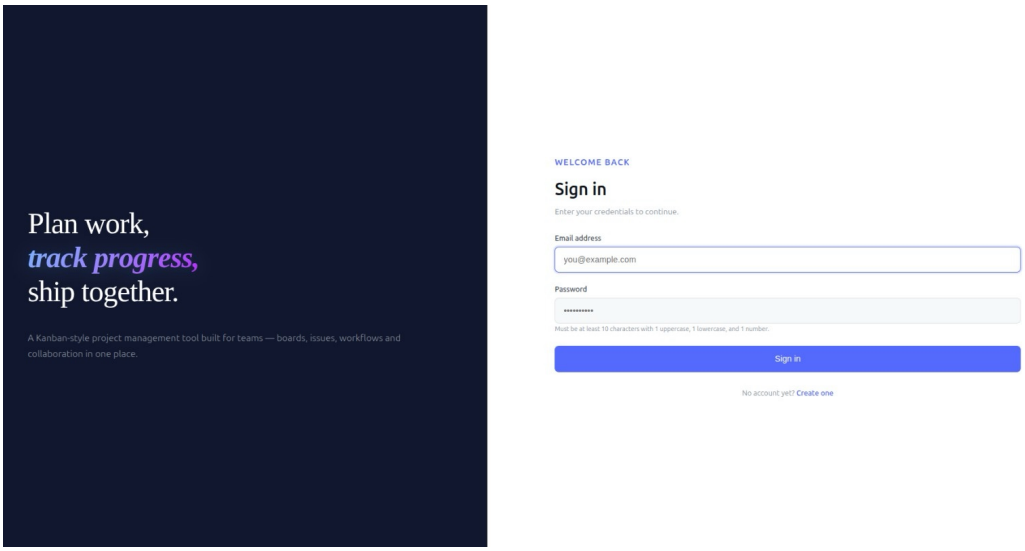

Figure 1: Login Page

Design Decisions:

- Split-screen layout with a dark motivational panel on the left and a clean form on the right, creating visual hierarchy and providing context about the application.
- The left panel displays the app value proposition: "Plan work, track progress, ship together" — immediately communicating purpose.

- Password strength hint shown below the input field to guide users on the 10-character minimum requirement.
- Minimal form fields reduce friction for returning users.

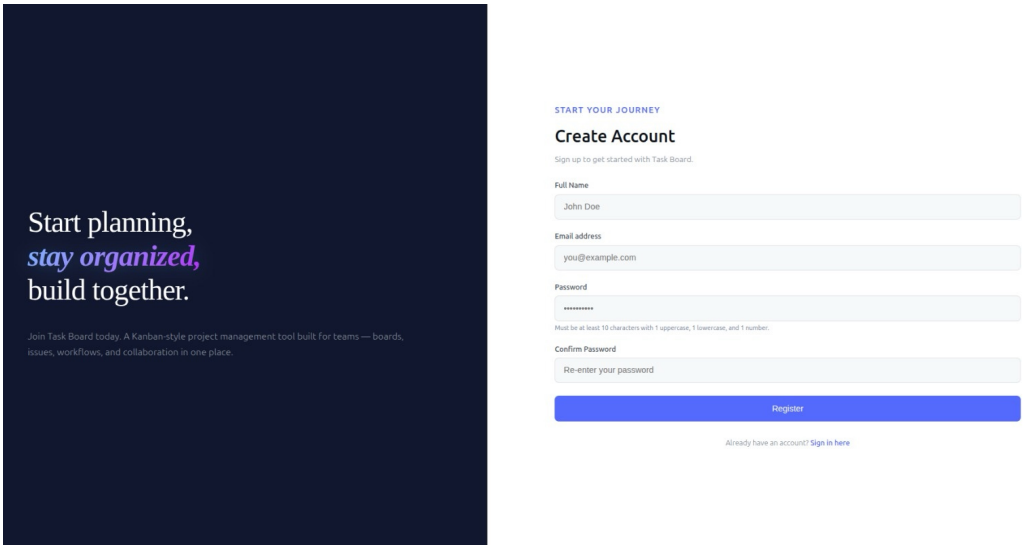

Figure 2: Registration Page

Design Decisions:

- Same split-screen layout as login for visual consistency across auth flows.
- Confirm Password field to prevent accidental typos during registration.
- Password requirements displayed inline rather than hidden in a tooltip.
- Clear navigation links between login and register reduce user confusion.

2.2 Dashboard & Home Page

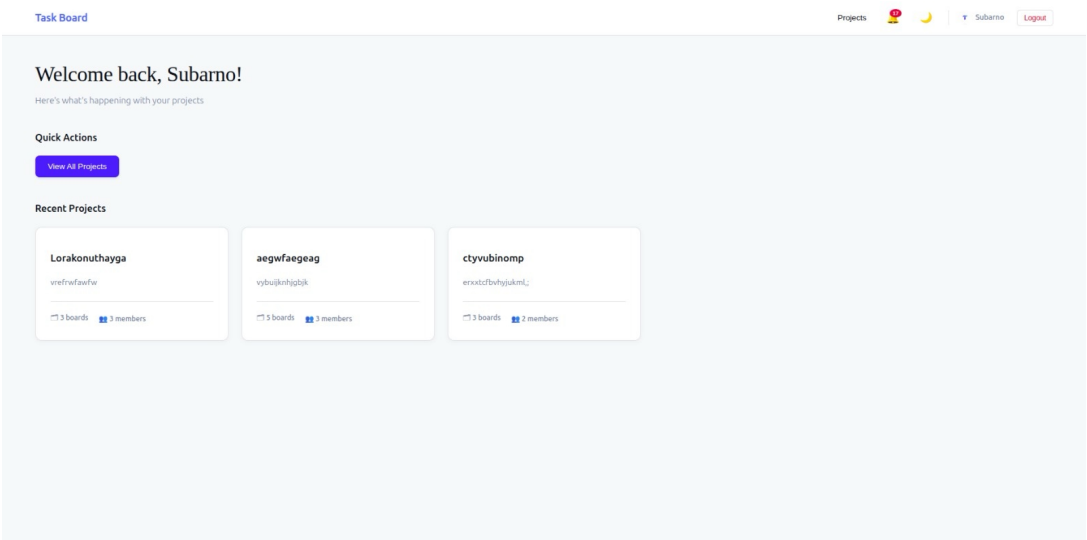

Figure 3: Home Dashboard

Design Decisions:

- Personalised greeting creates a warm, engaging experience for users.
- Quick Actions section with a prominent button for fast navigation.
- Recent Projects displayed as cards with board and member counts — giving users a snapshot of activity without drilling down.
- Notification bell in the navbar with unread count badge for immediate awareness.
- Dark mode toggle in the navbar respects user preference.

2.3 Projects List Page

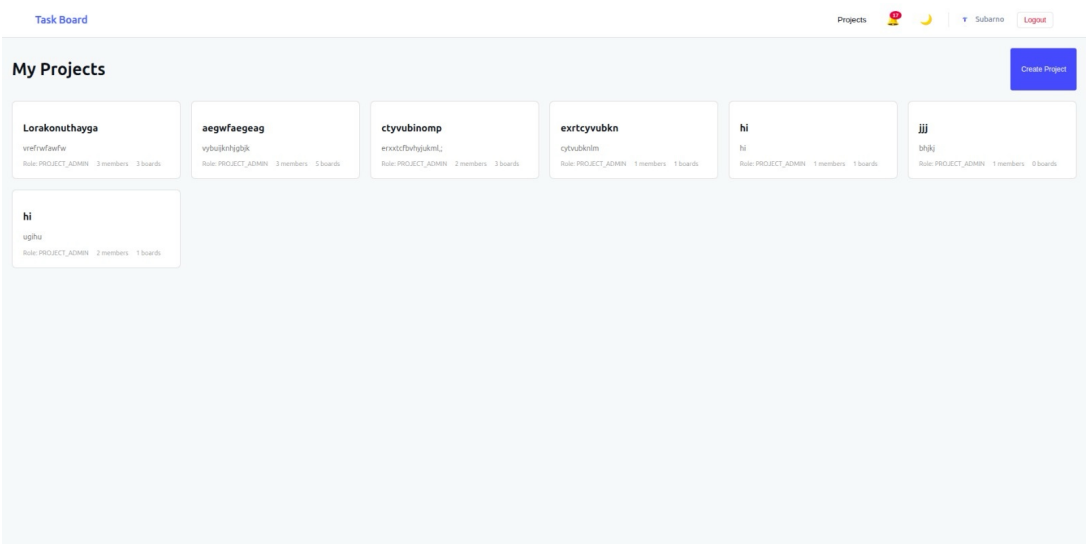

Figure 4: My Projects Page (Light Mode)

Design Decisions:

- Card grid layout allows quick scanning of multiple projects simultaneously.
- Each card shows project name, description, user role, member count, and board count — all critical information at a glance.
- Create Project button placed top-right following conventional UI placement.
- Role displayed on each card so users immediately know their permissions within that project.

2.4 Project Detail Page

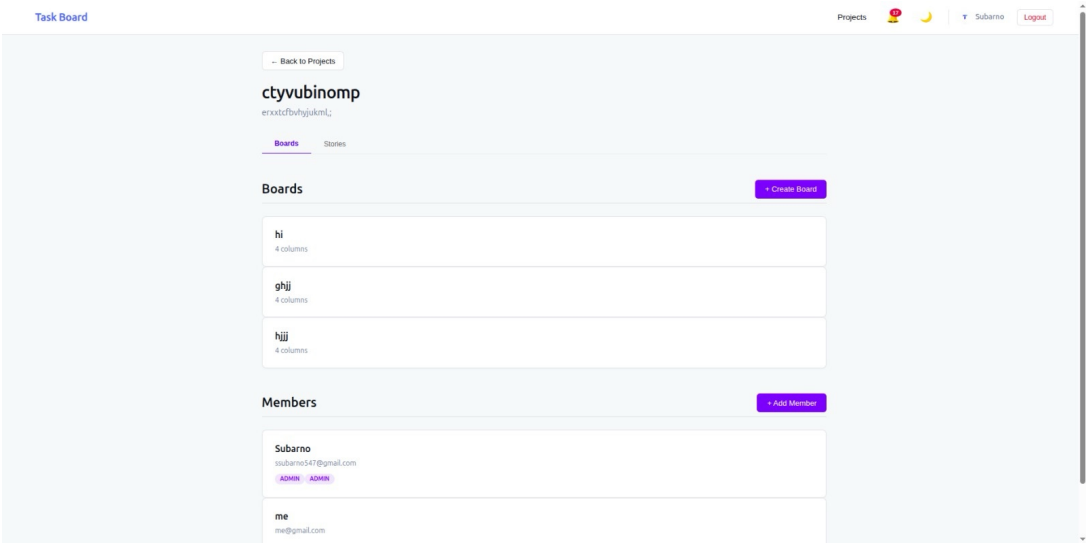

Figure 5: Project Detail — Boards Tab

Design Decisions:

- Tab navigation (Boards / Stories) allows switching between related views without navigating away, keeping context intact.
- Boards listed as full-width cards with column count, making it easy to identify board complexity.
- Members section with email, role tags and admin controls for project management.
- Add Member opens an inline form with email search rather than a separate page, reducing navigation overhead.
- Role tags shown as coloured badges for quick visual identification.

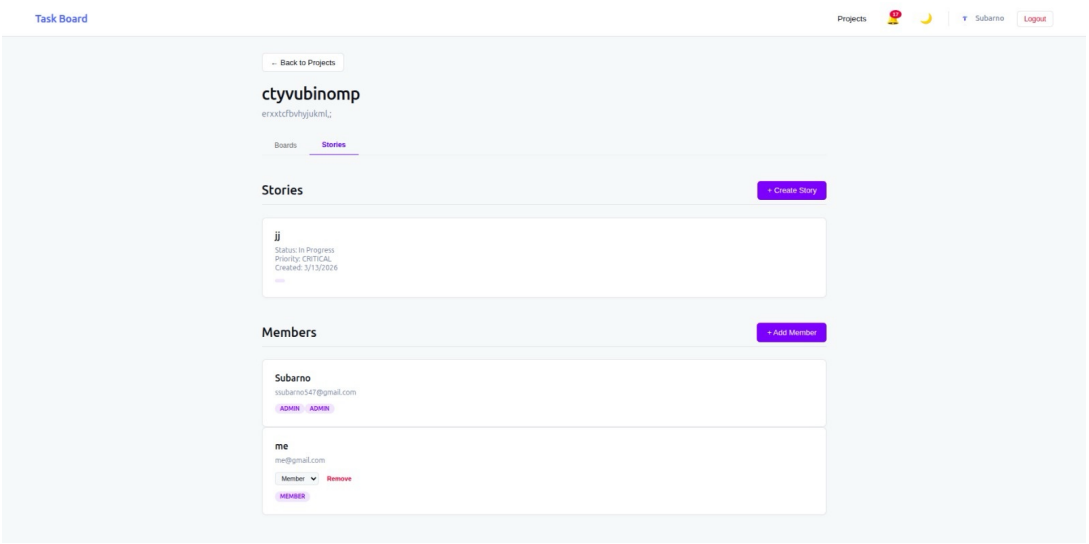

Figure 6: Project Detail — Stories Tab

Design Decisions:

- Stories tab is separate from Boards because stories are project-level concepts that span multiple boards, while boards are sprint-specific.
- Each story card shows status, priority, creation date and child task count.
- Create Story button allows direct story creation from the project level without opening a board first.

2.5 Board / Kanban View

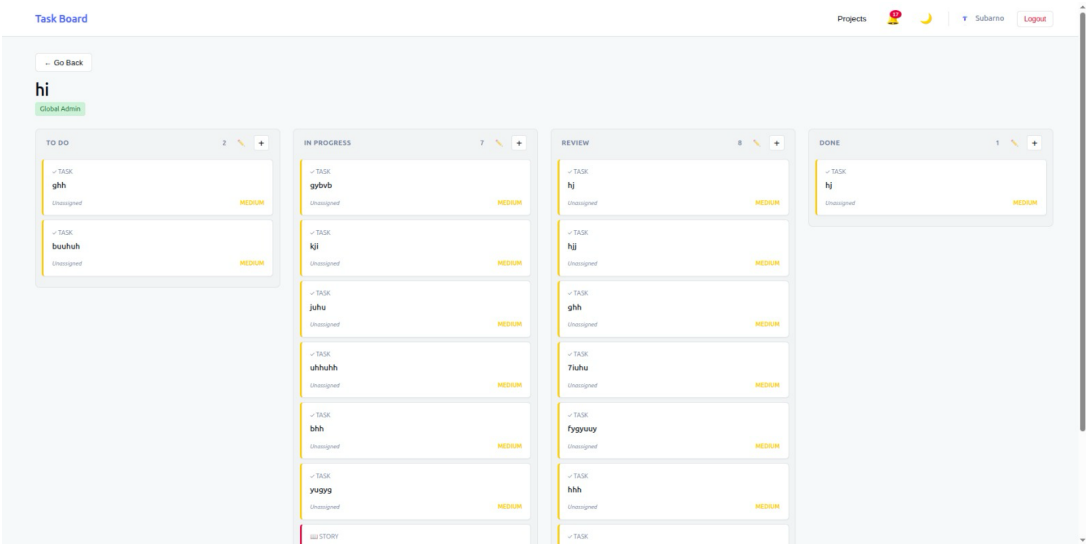

Figure 7: Kanban Board View

Design Decisions:

- Four default columns (To Do, In Progress, Review, Done) matching standard software development workflows.
- Issue count displayed in each column header for WIP visibility.
- Column header edit and add issue controls inline for quick access.
- Issue cards show type badge (TASK/BUG/STORY), title, assignee, and priority colour-coded.
- Global Admin badge shown on board header when user has admin privileges.
- Native HTML5 drag-and-drop used as per assignment requirements (no external libraries).

2.6 Issue Detail Modal

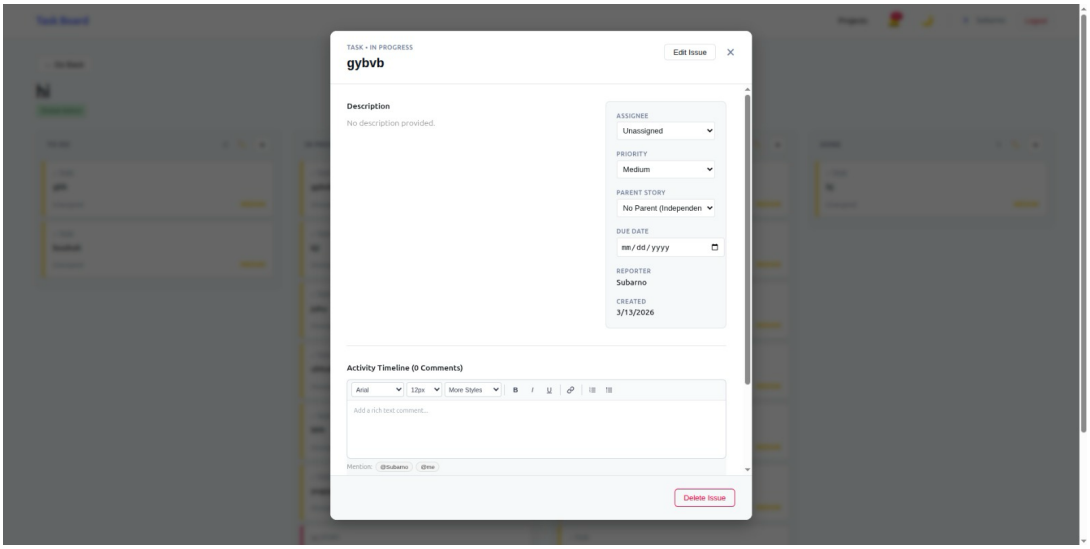

Figure 8: Issue Detail Modal — Task

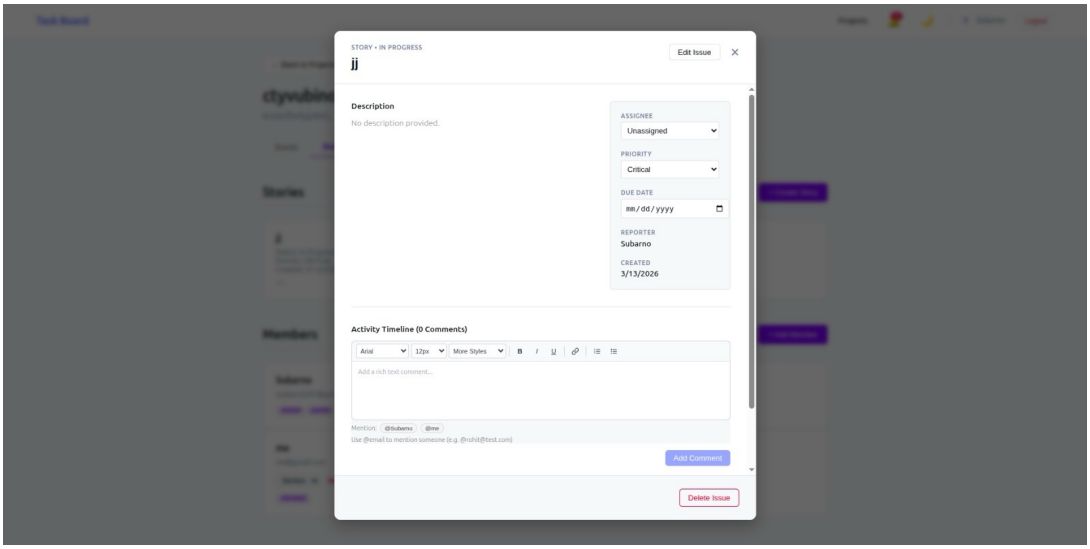

Figure 9: Issue Detail Modal — Story

Design Decisions:

- Modal overlay rather than a separate page keeps the board context visible in the background.
- Two-column layout: description and activity timeline on the left, metadata on the right.
- Rich text editor (react-quill-new) for description and comments, supporting bold, italic, lists, and links.
- Assignee dropdown populated with project members to prevent invalid assignments.
- Parent Story dropdown for tasks and bugs, enabling hierarchical issue organisation.

- @mention system: clickable member buttons below the comment box insert @email mentions that trigger notifications.
- Activity Timeline shows comments and audit log entries in chronological order.

2.7 User Profile Page

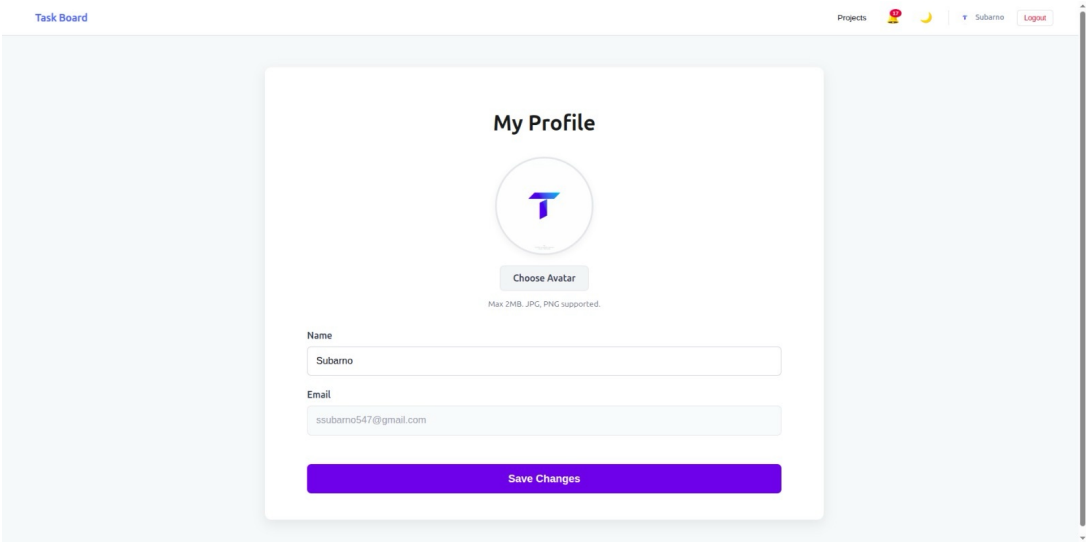

Figure 10: User Profile Page

Design Decisions:

- Avatar upload using base64 encoding — no external file storage service required, keeping the system self-contained.
- Email field displayed but disabled — email is the primary identifier and should not be changed.
- Avatar preview updates immediately on file selection before saving.
- File size limit (2MB) and format validation (JPG, PNG) shown below the upload button.
- Profile accessible by clicking the username in the navbar for intuitive navigation.

2.8 Notification Centre

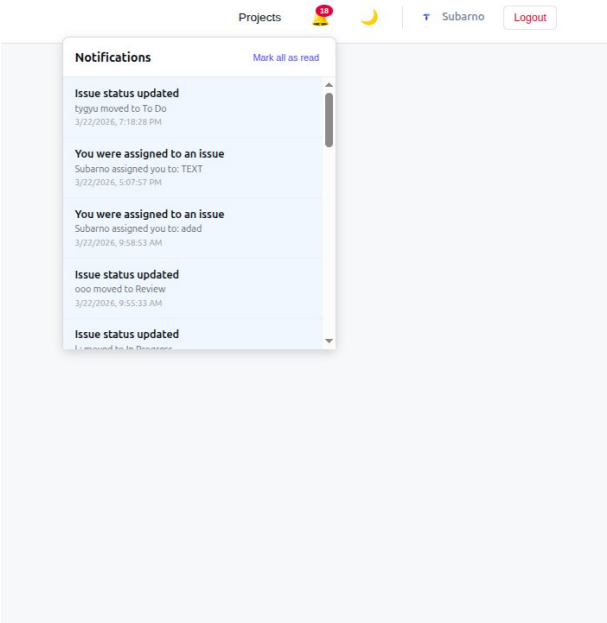

Figure 12: Notification Centre Dropdown

Design Decisions:

- Bell icon in the navbar shows a red unread count badge — immediately visible without opening the panel.
- Clicking the bell opens a scrollable dropdown listing all notifications with title, message, and timestamp.
- **"Mark all as read"** button at the top clears all unread badges in one action.
- Each notification shows a clear title (e.g. **"Issue status updated"**, **"You were assigned to an issue"**) and a short descriptive message below it.
- Timestamps shown on each notification for traceability and audit awareness.
- Notifications are displayed newest-first for immediate visibility of recent activity.
- Both light and dark modes are fully supported in the notification panel.
- Notifications are persistent — stored in the database and available across sessions.

2.9 Dark Mode

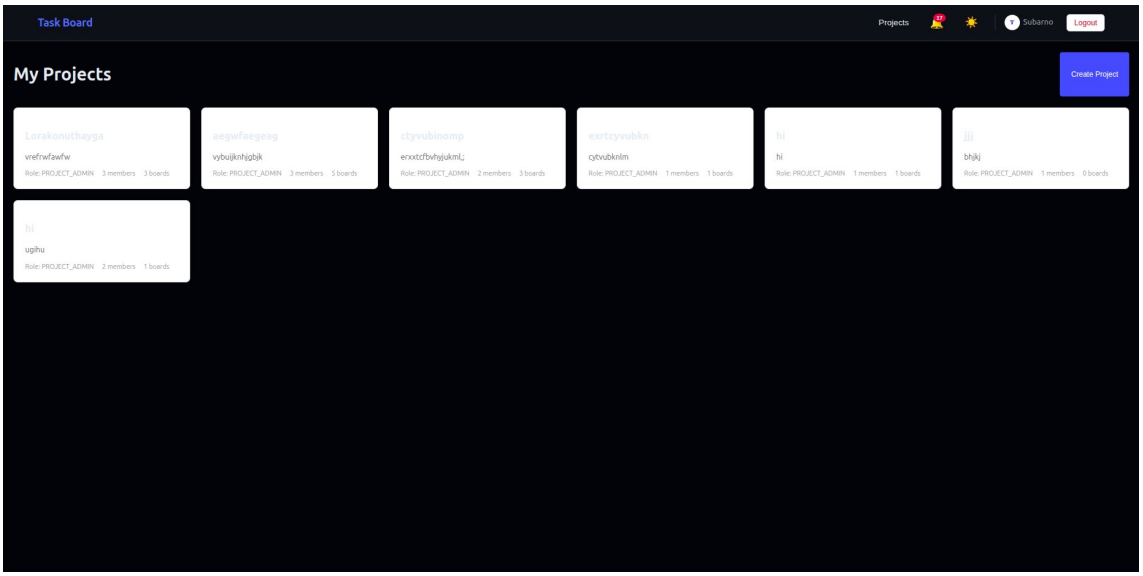

Figure 11: Projects Page — Dark Mode

Design Decisions:

- Dark mode implemented using CSS custom properties (`data-theme` on `document.documentElement`), enabling instant switching without page reload.
- Theme preference persisted in `localStorage` so users retain their choice across sessions.
- All components use CSS variables to ensure consistent theming across the entire application.

3 Backend Architecture Decisions

3.1 Authentication Design

- Dual token strategy: Short-lived access tokens (15 min) and long-lived refresh tokens (7 days) stored as HTTP-only cookies. This balances security with usability.
- JWT with JTI (JWT ID): Each token has a unique ID stored in the database, enabling true token invalidation on logout.
- Password hashing with bcrypt (salt rounds = 10) for secure storage.

3.2 Role-Based Access Control — Member-wise Roles

The system implements a two-level RBAC model. The table below details the exact permissions for each role:

Role	Permissions
Global Admin (globalRole)	• Create projects • Assign and modify WIP limits on any board column • Manage all users system-wide • Assign project-level roles to any user • Full access to all project settings • Cannot be edited or removed by Project Admins
Project Admin (PROJECT_ADMIN)	• Full access to project settings • Assign and modify WIP limits on board columns • Manage project members (add, remove, change roles) • Edit board workflows (add, rename, reorder, delete columns) • Create, edit, delete all issues • Cannot edit or remove a Global Admin member

Role	Permissions
Project Member (PROJECT_MEMBER)	• Create and edit issues (tasks, bugs, stories) • Move issues between columns • Add and delete own comments • Participate in project activity
Project Viewer (PROJECT_VIEWER)	• Read-only access to project, boards, and all issues • Can view comments and activity timeline • Cannot create, edit, move, or delete anything

Role Hierarchy Summary

- **Global Admin** — create projects, **assign WIP limits**, manage all users, assign project-level roles
  - **Project Admin** — full project settings, **assign WIP limits**, manage members, edit workflows
  - **Project Member** — create and edit tasks, participate in project
  - **Project Viewer** — read-only access to project and tasks

3.3 WIP Limits

- WIP limits enforced at both issue creation and issue movement time.
- Backend rejects operations exceeding the limit with a descriptive error message.
- WIP limits are optional per column — columns without limits have no restrictions.

WIP Limit Assignment

Both Global Admins and Project Admins can set and modify WIP limits on Kanban board columns. WIP limits are enforced at the API level — when a column has reached its limit, any attempt to move an additional issue into it is rejected with a **400 error**, not just a warning. Columns without a WIP limit set have no restriction.

3.4 Status Transition Validation

- Valid transitions enforced server-side: To Do → In Progress → Review → Done (with back-steps allowed).
- Direct jumps (e.g., To Do → Done) are blocked with a **400 error** explaining valid options.

3.5 Story Auto-Status Derivation

- When a child task or bug moves columns, the parent story status is automatically updated.
- All children Done → Story becomes Done. Any child In Progress/Review → Story becomes In Progress.

3.6 Notification System

- In-app notifications stored in the database and polled every 30 seconds from the frontend.
- Four types: `ASSIGNED`, `STATUS_CHANGED`, `COMMENT_ADDED`, `MENTIONED`.
- @mentions use email-based detection since usernames are not unique in this system.

4 Database Design

- UUID primary keys for all entities to avoid sequential ID enumeration attacks.
- Soft relationships: Issues have optional `parentId` for story-task hierarchy, with validation in the service layer.
- Audit log table: Every status change and assignee change recorded with old value, new value, user, and timestamp.
- Timestamps: All models have `createdAt` and `updatedAt`. Issues additionally track `resolvedAt` and `closedAt`.

- Prisma ORM for type-safe database access, automatic migrations, and connection pooling.

5 Testing Strategy

The project includes over 100 backend tests across 16 test suites covering all critical API endpoints and business logic.

Test Suite	Tests	Coverage
auth.test.ts	9	Register, login, profile, tokens
project.test.ts	4	Create, get, auth checks
project_management.test.ts	10	Update, archive, delete, access control
board.test.ts	4	Create with default columns, get
board_management.test.ts	10	Update, delete, column CRUD
issue.test.ts	7	CRUD, status transitions, move validation
comment.test.ts	3	Add, get, validation
notification.test.ts	5	Get, unread count, mark as read
user.test.ts	7	Profile, update, search, avatar
wip.test.ts	3	WIP limit enforcement
story.test.ts	5	Parent-child, auto-status derivation
member.test.ts	6	Add, remove, role update, access control
audit.test.ts	4	Audit log on changes
integration.test.ts	6	Full workflow end-to-end
integration2.test.ts	12	Edge cases, security, multi-user
simple.test.ts	24	Comprehensive API coverage
Total	**100+**	**All critical endpoints**

Table 2: Test Coverage Summary

6 Setup Instructions

6.1 Prerequisites

Node.js v18+, PostgreSQL 16+, npm

6.2 Backend Setup

```
cd backend
npm install
# Create .env with DATABASE_URL, JWT secrets
npx prisma db push
npm run dev
```

6.3 Frontend Setup

```
cd frontend
npm install
# Create .env with VITE_API_URL=http://localhost:3000/api
npm run dev
```

6.4 Running Tests

```
cd backend
# Ensure taskboard_test database exists
npm test -- --runInBand
```

GitHub: https://github.com/88subarno88/Task_Board